Advanced OOP

Practical-1

CASE STUDY: BANK ACCOUNT AND MONEY TRANSFERS

2520030032

A. Amulya

1. Money.java

```
class Money {
    private final long amountInPaise;
    private final String currency;

    // Constructor
    Money(long amountInPaise, String currency) {
        if (amountInPaise < 0) {
            throw new IllegalArgumentException("Amount cannot be negative");
        }
        this.amountInPaise = amountInPaise;
        this.currency = currency;
    }

    // Addition
    Money add(Money other) {

        return new Money(
            this.amountInPaise + other.amountInPaise,
            currency
        );
    }
```

```
// Subtraction
Money subtract(Money other) {

    if (this.amountInPaise < other.amountInPaise) {
        throw new IllegalArgumentException("Insufficient balance");
    }

    return new Money(
        this.amountInPaise - other.amountInPaise,
        currency
    );
}

// Check positive amount
boolean isPositive() {
    return amountInPaise > 0;
}

// Getters
long getAmountInPaise() {
    return amountInPaise;
}

String getCurrency() {
    return currency;
}

public String toString() {
```

```
        return currency + " " + (amountInPaise / 100.0);
    }
}
```

2. AccountStatus.java

```
enum AccountStatus {

    ACTIVE,
    FROZEN,
    CLOSED

}
```

3. BankAccount.java

```
class BankAccount {

    private final String accountNumber;
    private Money balance;
    private final Money overdraftLimit;
    private AccountStatus status;

    // Constructor
    BankAccount(String accountNumber, Money overdraftLimit) {

        this.accountNumber = accountNumber;
        this.overdraftLimit = overdraftLimit;

        balance = new Money(0, "INR");

        status = AccountStatus.ACTIVE;
```

```
}

// Deposit
void deposit(Money amount) {

    if (status != AccountStatus.ACTIVE) {
        System.out.println("Account is not active");
        return;
    }

    if (!amount.isPositive()) {
        System.out.println("Invalid amount");
        return;
    }

    balance = balance.add(amount);

    System.out.println("Deposit successful");
}

// Withdraw
void withdraw(Money amount) {

    if (status != AccountStatus.ACTIVE) {
        System.out.println("Account is not active");
        return;
    }

    if (!amount.isPositive()) {
```

```
            System.out.println("Invalid amount");
            return;
        }

        long availableBalance =
            balance.getAmountInPaise() +
            overdraftLimit.getAmountInPaise();

        if (amount.getAmountInPaise() > availableBalance) {
            System.out.println("Overdraft limit exceeded");
            return;
        }

        balance = new Money(
            balance.getAmountInPaise() -
            amount.getAmountInPaise(),
            "INR"
        );

        System.out.println("Withdrawal successful");
    }

    // Freeze
    void freeze() {

        status = AccountStatus.FROZEN;

        System.out.println("Account frozen");
    }
```

```
// Unfreeze
void unfreeze() {

    if (status == AccountStatus.FROZEN) {

        status = AccountStatus.ACTIVE;

        System.out.println("Account unfrozen");
    }
}

// Close
void close() {

    if (balance.getAmountInPaise() == 0) {

        status = AccountStatus.CLOSED;

        System.out.println("Account closed");
    }
    else {
        System.out.println("Balance must be zero to close account");
    }
}

// Get balance
Money getBalance() {
    return balance;
```

```
    }

    String getAccountNumber() {
        return accountNumber;
    }

    AccountStatus getStatus() {
        return status;
    }
}
```

4. LedgerEntry.java

```
import java.time.LocalDateTime;

class LedgerEntry {

    private final String transactionId;
    private final String transactionType;
    private final Money amount;
    private final String timestamp;

    // Constructor
    LedgerEntry(String transactionId,
                String transactionType,
                Money amount) {

        this.transactionId = transactionId;
        this.transactionType = transactionType;
```

```
    this.amount = amount;

    timestamp = LocalDateTime.now().toString();
}

// Getters
String getTransactionId() {
    return transactionId;
}

String getTransactionType() {
    return transactionType;
}

Money getAmount() {
    return amount;
}

String getTimestamp() {
    return timestamp;
}

// Display transaction
void display() {

    System.out.println("Transaction ID: " + transactionId);
    System.out.println("Transaction Type: " + transactionType);
    System.out.println("Amount: " + amount);
    System.out.println("Timestamp: " + timestamp);
```

```
    }
}
```

5. FundTransfer.java

```
class FundTransfer {

    private final String transferId;
    private final BankAccount source;
    private final BankAccount destination;
    private final Money amount;

    // Constructor
    FundTransfer(String transferId,
                 BankAccount source,
                 BankAccount destination,
                 Money amount) {

        this.transferId = transferId;
        this.source = source;
        this.destination = destination;
        this.amount = amount;
    }

    // Transfer method
    void transfer() {

        if (source.getStatus() != AccountStatus.ACTIVE ||
            destination.getStatus() != AccountStatus.ACTIVE) {
```

```
    System.out.println("Both accounts must be active");
    return;
}

if (!amount.isPositive()) {
    System.out.println("Invalid transfer amount");
    return;
}

if (!source.getBalance().getCurrency().equals(
    destination.getBalance().getCurrency())) {

    System.out.println("Currency mismatch");
    return;
}

// Withdraw from source
long oldSourceBalance =
    source.getBalance().getAmountInPaise();

source.withdraw(amount);

// Check whether withdrawal succeeded
if (source.getBalance().getAmountInPaise() ==
    oldSourceBalance - amount.getAmountInPaise()) {

    // Deposit into destination
    destination.deposit(amount);
```

```
            System.out.println("Transfer successful");
            System.out.println("Transfer ID: " + transferId);
        }
        else {
            System.out.println("Transfer failed");
        }
    }
}
```

6. Main.java

```
public class Main {

    public static void main(String[] args) {

        // Create overdraft limit
        Money limit = new Money(500000, "INR");

        // Create accounts
        BankAccount a1 = new BankAccount("1001", limit);
        BankAccount a2 = new BankAccount("1002", limit);

        // Deposit
        System.out.println("---- DEPOSIT ----");

        a1.deposit(new Money(1000000, "INR"));
        a2.deposit(new Money(500000, "INR"));

        System.out.println("Account 1: " + a1.getBalance());
        System.out.println("Account 2: " + a2.getBalance());
```

```
// Withdraw
System.out.println("\n---- WITHDRAW ----");

a1.withdraw(new Money(200000, "INR"));

System.out.println("Balance: " + a1.getBalance());

// Transfer
System.out.println("\n---- FUND TRANSFER ----");

FundTransfer f1 = new FundTransfer(
    "T101",
    a1,
    a2,
    new Money(300000, "INR")
);

f1.transfer();

System.out.println("Account 1: " + a1.getBalance());
System.out.println("Account 2: " + a2.getBalance());

// Freeze
System.out.println("\n---- FREEZE ----");

a1.freeze();

a1.withdraw(new Money(100000, "INR"));
```

```
        // Unfreeze
        System.out.println("\n---- UNFREEZE ----");

        a1.unfreeze();

        // Ledger
        System.out.println("\n---- LEDGER ----");

        LedgerEntry l1 = new LedgerEntry(
            "L101",
            "DEPOSIT",
            new Money(1000000, "INR")
        );

        l1.display();

        // Close account demonstration
        System.out.println("\n---- ACCOUNT STATUS ----");

        System.out.println("Account 1: " + a1.getStatus());
        System.out.println("Account 2: " + a2.getStatus());
    }
}
```

Output:

```
PS D:\AOOP\Practical\Practical-1>  & 'C:\Users\amuly\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:51840' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\amuly\AppData\Roaming\Code\User\workspaceStorage\99242b0cd37fb4f3c4a2e17e184e1b6f\redhat.java\jdt_ws\Practical-1_cacf8af8\bin' 'Main'
---- DEPOSIT ----
Deposit successful
Deposit successful
Account 1: INR 10000.0
Account 2: INR 5000.0

---- WITHDRAW ----
Withdrawal successful
Balance: INR 8000.0

---- FUND TRANSFER ----
Withdrawal successful
Deposit successful
Transfer successful
Transfer ID: T101
Account 1: INR 5000.0
Account 2: INR 8000.0

---- FREEZE ----
Account frozen
Account is not active

---- UNFREEZE ----
Account unfrozen

---- LEDGER ----
Transaction ID: L101
Transaction Type: DEPOSIT
Amount: INR 10000.0
Timestamp: 2026-11-11T11:24:36.407727

---- ACCOUNT STATUS ----
Account 1: ACTIVE
Account 2: ACTIVE
PS D:\AOOP\Practical\Practical-1>
```